

Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №1
На тему
“Основи функціонального програмування у Python”

Виконав:
Студент групи ФЕП-21
Бардак Д. Т

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати `typing.Callable` для параметризації обчислень

Теоретичні відомості:

- Чиста функція: детермінована (однаковий вхід $\rightarrow$ однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
- Референтна прозорість: вираз можна замінити його значенням без зміни поведінки програми.
- Побічні ефекти: I/O (`print`, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від `time`, `random`, `os.environ` тощо.
- Підхід: відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Хід роботи

У ході виконання лабораторної роботи було розроблено програму для обробки замовлень із суворим розділенням чистого обчислювального ядра та оболонки побічних ефектів. У файлі `core.py` реалізовано чисті функції, які відповідають за математичні розрахунки та створення нових об'єктів без мутації вхідних структур. Для параметризації обчислень було застосовано тип `typing.Callable`, що дозволило гнучко налаштовувати політики знижок, податків та фільтрації. Усі операції вводу-виводу, зокрема виведення результатів на екран, було повністю ізолювано у файлі `app.py`. Для підтвердження надійності коду у файлі `test_core.py` було написано автоматизовані тести з використанням фреймворку `pytest`, які успішно перевірили референтну прозорість, відсутність мутацій оригінальних даних та коректність роботи переданих функцій-політик. Фінальна перевірка продемонструвала безпомилкове виконання програми, проходження всіх тестів та відсутність проблем із типізацією за результатами аналізу туру.

```

1  from collections.abc import Callable, Iterable
2  from typing import TypedDict
3
4
5  class Item(TypedDict):
6      price: float
7      qty: int
8
9
10 class Order(TypedDict, total=False):
11     id: int
12     items: list[Item]
13     paid: bool
14     total: float
15
16
17 FilterFn = Callable[[float], bool]
18 DiscountFn = Callable[[float], float]
19 TaxFn = Callable[[float], float]
20 ProcessorFn = Callable[[Iterable[Order]], dict[str, object]]
21
22
23 def order_subtotal(order: Order) -> float:
24     return sum(it["price"] * it["qty"] for it in order.get("items", []))
25
26
27 def with_total(order: Order, total: float) -> Order:
28     return {**order, "total": total}
29
30
31 def make_processor(
32     accept: FilterFn,
33     apply_discount: DiscountFn,
34     apply_tax: TaxFn,
35 ) -> ProcessorFn:
36     def process(orders: Iterable[Order]) -> dict[str, object]:
37         qualified = []
38         revenue = 0.0
39
40         for o in orders:
41             if not o.get("paid"):
42                 continue
43
44             subtotal = order_subtotal(o)
45
46             if not accept(subtotal):
47                 continue
48
49             total = apply_tax(apply_discount(subtotal))
50
51             new_order = with_total(o, total)
52             qualified.append(new_order)
53             revenue += total
54
55         return {"count": len(qualified), "revenue": revenue, "orders": qualified}
56
57     return process

```

Рис. 1-3 Файл core.py

```

1  from typing import cast
2
3  from core import Order, make_processor
4
5
6  def render_report(result: dict[str, object]) -> None:
7      orders = cast(list[Order], result.get("orders", []))
8
9      print("=== ЗВІТ З ОБРОБКИ ЗАМОВЛЕНЬ ===")
10     for o in orders:
11         print(f"Оброблено замовлення #{o['id']} | Підсумкова сума: {o['total']:.2f}")
12
13     print("-" * 30)
14     print(f"Кількість валідних замовлень: {result['count']}")
15
16     revenue = cast(float, result.get("revenue", 0.0))
17     print(f"Загальний дохід (Revenue): {revenue:.2f}")
18
19
20 def main() -> None:
21     sample_orders: list[Order] = [
22         {
23             "id": 1,
24             "items": [{"price": 50.0, "qty": 2}, {"price": 10.0, "qty": 1}],
25             "paid": True,
26         },
27         {
28             "id": 2,
29             "items": [{"price": 20.0, "qty": 1}],
30             "paid": False,
31         },
32         {
33             "id": 3,
34             "items": [{"price": 30.0, "qty": 2}],
35             "paid": True,
36         },
37         {"id": 4, "items": [{"price": 200.0, "qty": 1}], "paid": True}, # Сума: 200
38     ]
39
40     accept_policy = lambda s: s >= 100
41     discount_policy = lambda s: s * 0.9
42     tax_policy = lambda s: s * 1.2
43
44     processor = make_processor(
45         accept=accept_policy, apply_discount=discount_policy, apply_tax=tax_policy
46     )
47
48     result = processor(sample_orders)
49
50     render_report(result)
51
52
53 if __name__ == "__main__":
54     main()

```

Рис. 4-5 Файл app.py

```

1  from copy import deepcopy
2
3  import pytest
4
5  from core import Order, make_processor
6
7
8  @pytest.fixture
9  def sample_orders() -> list[Order]:
10     return [
11         {"id": 1, "items": [{"price": 100.0, "qty": 1}], "paid": True},
12         {"id": 2, "items": [{"price": 50.0, "qty": 2}], "paid": True},
13     ]
14
15
16  def test_referential_transparency(sample_orders: list[Order]) -> None:
17     processor = make_processor(
18         accept=lambda s: s >= 0,
19         apply_discount=lambda s: s * 0.9,
20         apply_tax=lambda s: s * 1.2,
21     )
22
23     r1 = processor(sample_orders)
24     r2 = processor(sample_orders)
25
26     assert r1 == r2
27
28
29  def test_no_mutation(sample_orders: list[Order]) -> None:
30     original = deepcopy(sample_orders)
31     processor = make_processor(
32         accept=lambda s: s >= 0, apply_discount=lambda s: s, apply_tax=lambda s: s
33     )
34
35     processor(sample_orders)
36
37     assert sample_orders == original
38
39
40  def test_callable_policies(sample_orders: list[Order]) -> None:
41     processor = make_processor(
42         accept=lambda s: s >= 100, # Тільки суми >= 100
43         apply_discount=lambda s: s - 10, # Фіксована знижка 10
44         apply_tax=lambda s: s * 1.1, # Податок 10%
45     )
46
47     result = processor(sample_orders)
48
49     assert result["count"] == 2
50
51     orders = result["orders"]
52     assert isinstance(orders, list)
53
54     assert abs(orders[0]["total"] - 99.0) < 0.001
55
56     assert result["revenue"] == pytest.approx(198.0)

```

Рис. 6-8 Файл test_core.py

```

/home/ritokk/Documents/fp/lab01/.venv/bin/python /home/ritokk/Documents/fp/lab01/lab01/app.py
=== ЗВІТ З ОБРОБКИ ЗАМОВЛЕНЬ ===
Оброблено замовлення #1 | Підсумкова сума: 118.80
Оброблено замовлення #4 | Підсумкова сума: 216.00
-----
Кількість валідних замовлень: 2
Загальний дохід (Revenue): 334.80

Process finished with exit code 0

```

Рис. 9 Запуск програми

```
(.venv) arch-laptop% python -m pytest tests/
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/ritokk/Documents/fp/lab01/lab01
collected 3 items

tests/test_core.py ... [100%]

===== 3 passed in 0.03s =====
(.venv) arch-laptop% mypy core.py app.py
Success: no issues found in 2 source files
(.venv) arch-laptop%
```

Рис.10 Успішно виконані тести

Висновок:

Під час виконання лабораторної роботи було успішно переписано імперативний фрагмент коду у функціональному стилі мовою Python. У межах завдання було здобуто практичні навички роботи з чистими функціями та ізоляції побічних ефектів шляхом відокремлення обчислювального ядра від операцій I/O. Завдяки застосуванню анотацій типів та Callable-об'єктів вдалося досягти високої гнучкості й референтної прозорості коду без використання глобального стану чи мутацій вхідних даних.